Systems Research Group
Department of Computer Science
Technical University of Munich

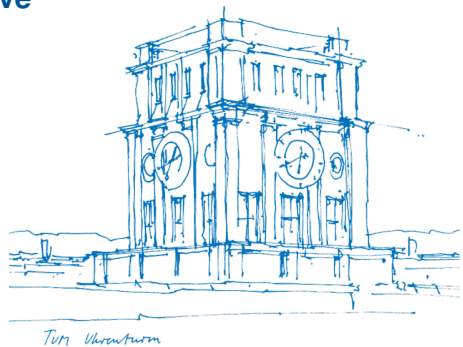
mininet

A Shared-Nothing HTTPS Stack for Cloud-Native Analytical Engines

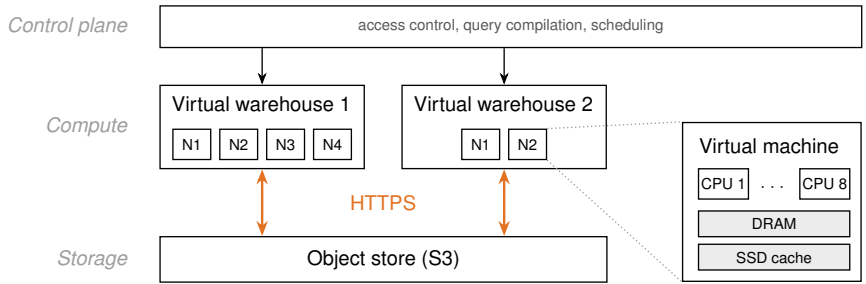
Martin Lindbüchl

Guided Research — Final Presentation
Advisor: Ilya Meignan--Masson
Systems Research Group

October 20th, 2026



Motivation
Resource Disaggregation



Snowflake, simplified after Dageville et al., SIGMOD'16

Motivation

Resource Disaggregation



Motivation

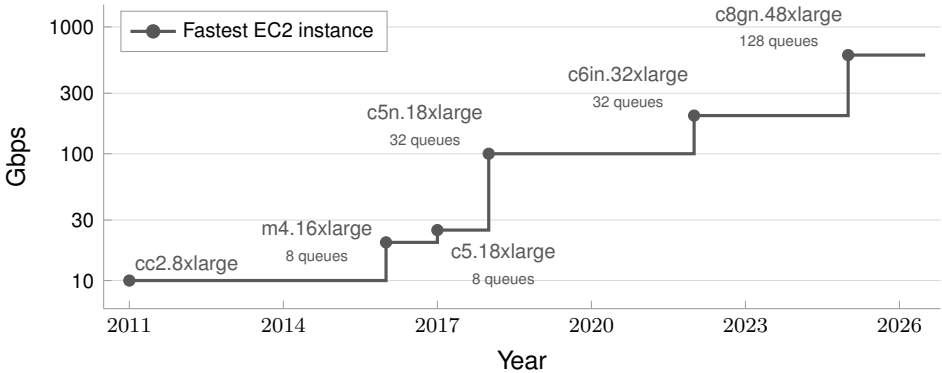
Resource Disaggregation



Requires high-bandwidth network stack optimized for client-workloads

Motivation

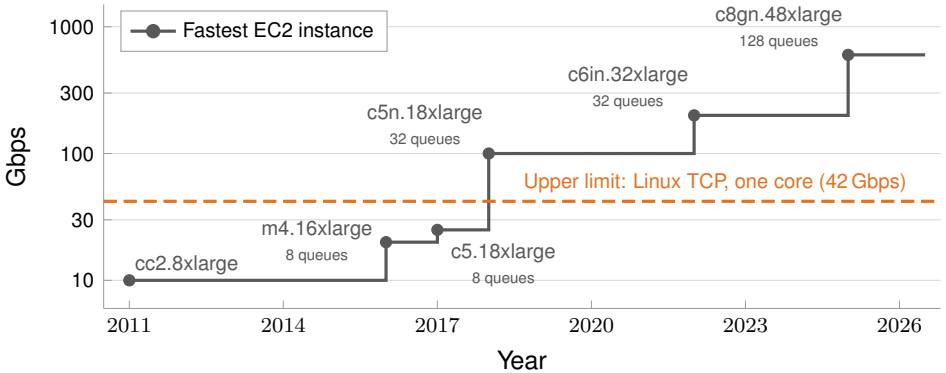
Network Bandwidth in the Cloud



60× bandwidth in 15 years

Motivation

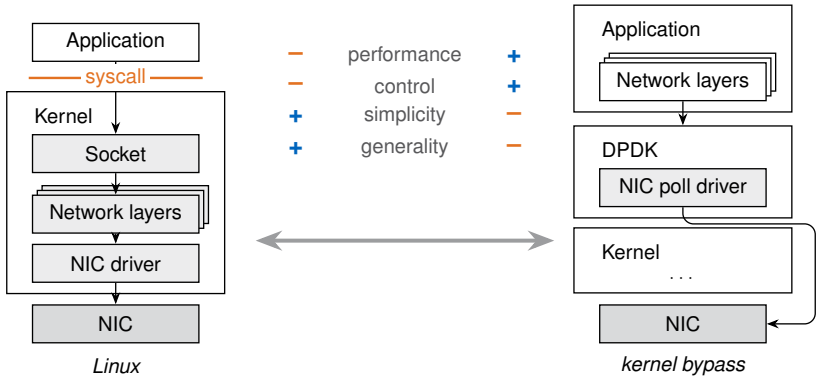
Network Bandwidth in the Cloud



Linux per-core bandwidth falls behind network bandwidth

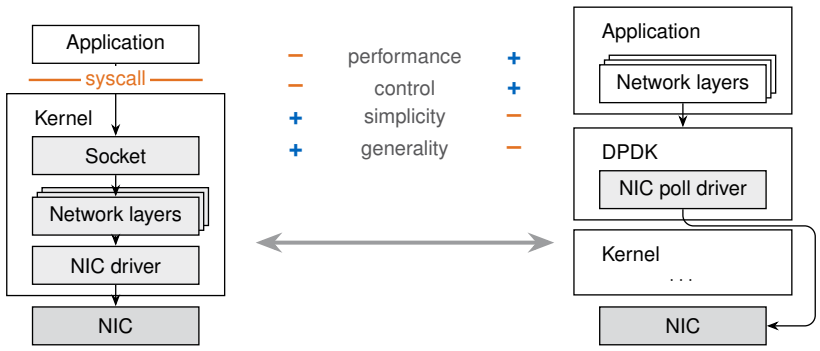
Motivation

Linux Networking and Kernel Bypass



Motivation

Linux Networking and Kernel Bypass



Can we overcome the limitations of each approach?

Problem Statement

Networking Inside Unikernels

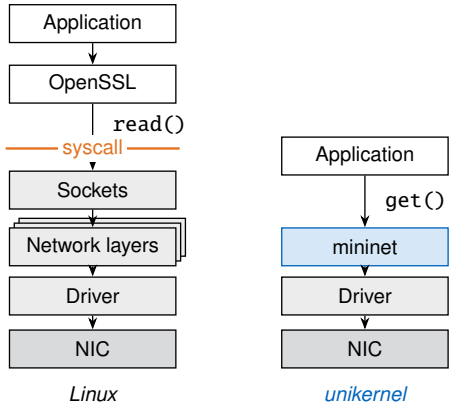
Can the OS provide a **simple**, **resource efficient**, **high bandwidth** network stack?

1. High bandwidth
2. Low CPU overhead
3. Easy to integrate

Key Idea
Move past POSIX

A Unikernel Architecture

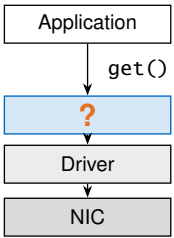
- Co-design network stack and kernel
  Network stack aware scheduling
- Benefit from a single address space
  No context switches or memory copies
- Remove unnecessary abstractions
  Integrate the stack into the application



Research Gap
Efficient Networking

Existing Unikernels

- Reuse a general-purpose OS stack
  OSv, UKL, Lupine
- Implement a stack that serializes packet processing
  MirageOS, IncludeOS
- Reuse a serial standalone stack
  Unikraft, RustyHermit



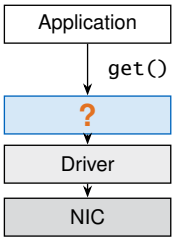
unikernel

Research Gap

Efficient Networking

Existing Unikernels

- Reuse a general-purpose OS stack
  OSv, UKL, Lupine
- Implement a stack that serializes packet processing
  MirageOS, IncludeOS
- Reuse a serial standalone stack
  Unikraft, RustyHermit

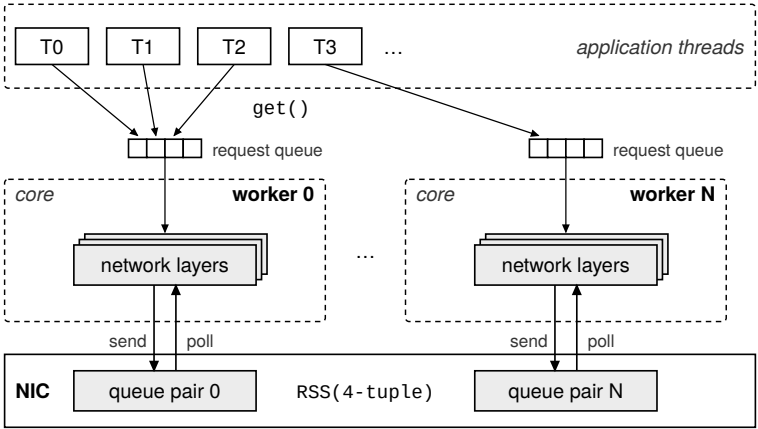


unikernel

None of these stacks meet the demands of analytical workloads

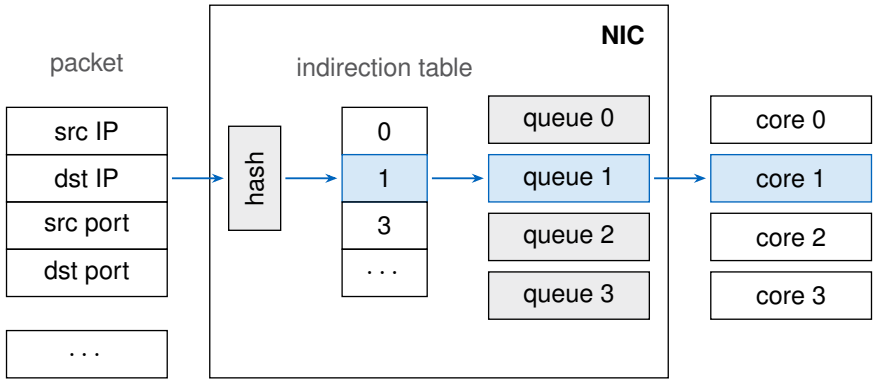
A Unikernel Network Stack

Design Overview



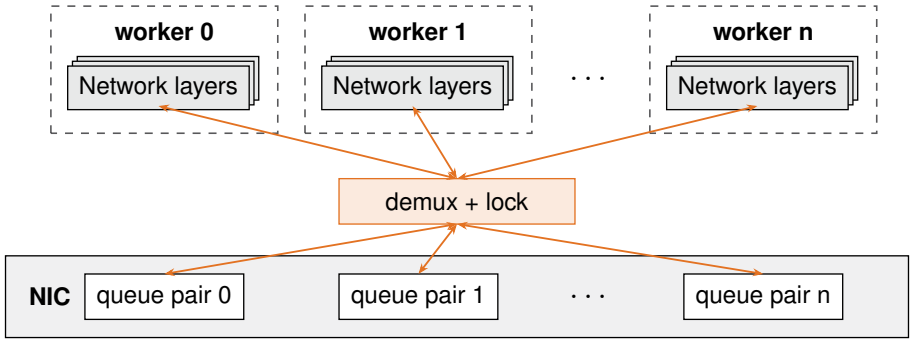
Receive Side Scaling

Background



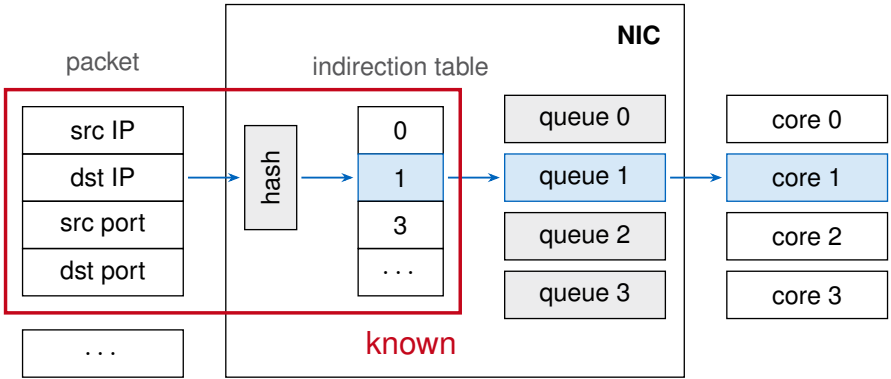
A Unikernel Network Stack

RX Path



Receive Side Scaling

Predictable Packet Steering

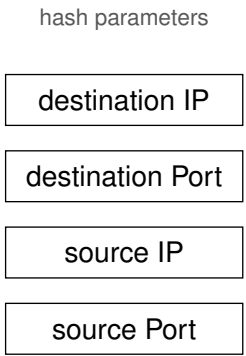


→ Steering decision only relies on information known before a request is sent

Receive Side Scaling
Leveraging Predictable Packet Steering

Workload

- Client network stack
- Long-lived TCP connections
- Few remote endpoints

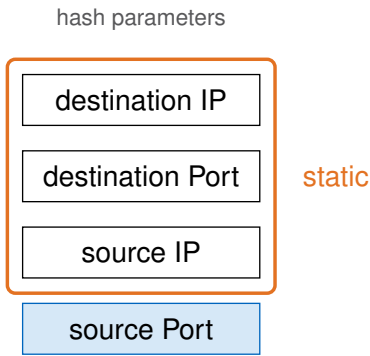


Receive Side Scaling

Leveraging Predictable Packet Steering

Workload

- Client network stack
- Long-lived TCP connections
- Few remote endpoints

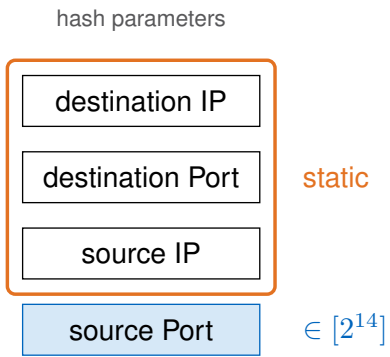


Receive Side Scaling
Leveraging Predictable Packet Steering

Workload

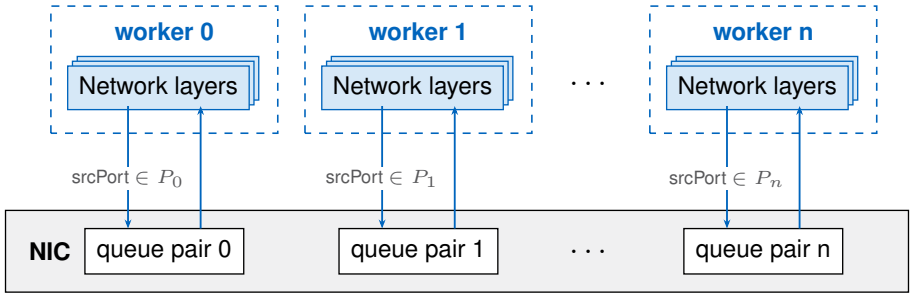
- Client network stack
- Long-lived TCP connections
- Few remote endpoints

Source port determines the reply's queue



A Unikernel Network Stack

Slicing the Stack



A Unikernel Network Stack

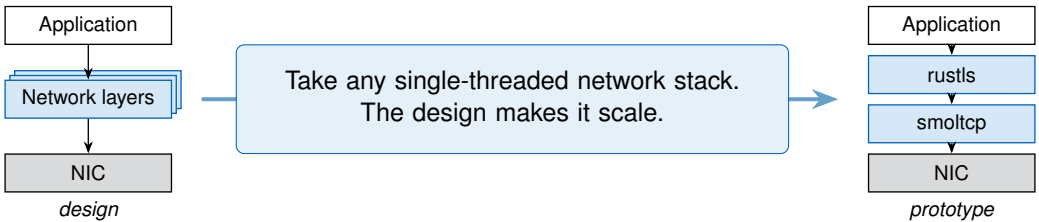
Implementation

smoltcp Ethernet, IP, TCP

- Standalone, device-agnostic Rust crate
- No runtime, no event loop, single-threaded
- Unmodified, one instance per worker

rustls TLS 1.3

- Pure-Rust TLS library
- Session state stays with the worker
- Unmodified, one instance per worker



Evaluation
Setup

Testbed

instance	vCPUs	memory	network
`r6in.8xlarge`	32	256 GiB	50 Gbps
`r6in.32xlarge`	128	1 TiB	200 Gbps

- Network-optimized AWS VMs
- One ENA NIC per VM
- Object store: S3 via DuckDB's httpfs

Competitors

- Linux kernel stack, GRO disabled
- anyblob [VLDB'23]: `io_uring` download manager

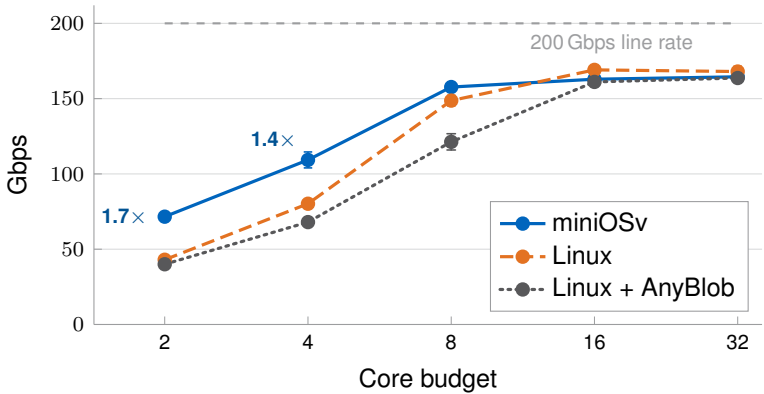
Benchmarks

- Bandwidth: bulk GETs from nginx
- TPC-H SF 100: DuckDB, parquet on S3

Evaluation
Bandwidth

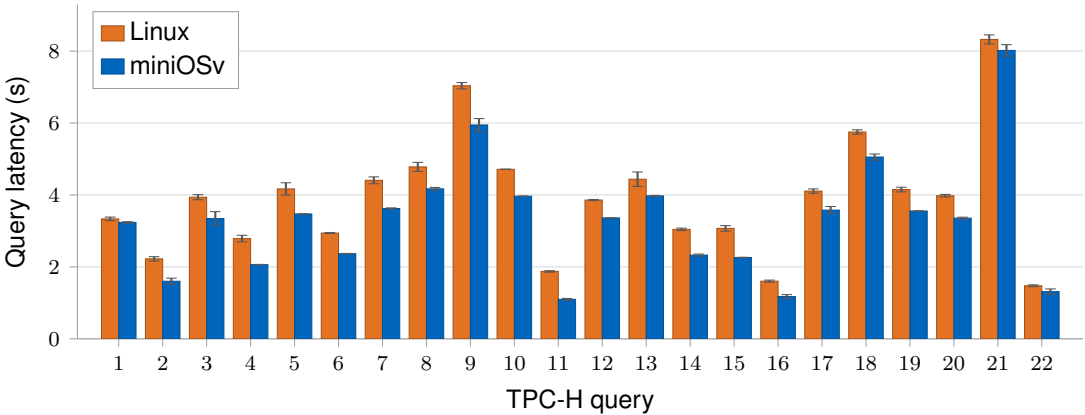
Workload

- pull from nginx
- 32 MiB ranged GETs
- HTTP, no TLS
- `r6in.32xlarge`



Evaluation

TPC-H

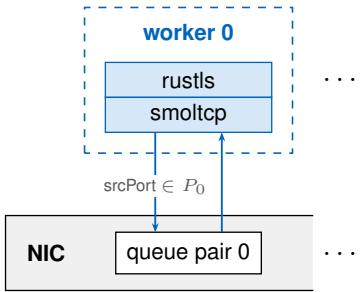


Conclusion
A Shared-Nothing HTTPS Stack

Can the OS provide a **simple**, **resource efficient**, **high bandwidth** network stack?

mininet

- Purpose-built, shared-nothing, network stack
- Easy integration into existing analytical engines
- 1.22× faster than Linux for TPC-H on DuckDB



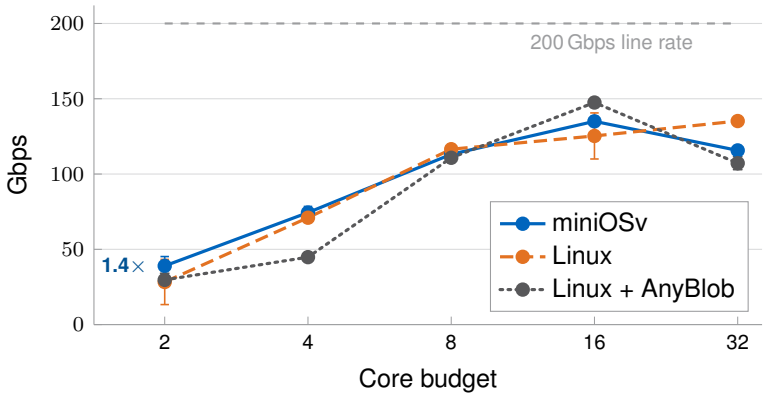
Backup Slides

Evaluation
HTTP GETs from S3

Workload

- pull from S3
- 64 connections
- 32 MiB ranged GETs
- HTTP, no TLS
- `r6in.32xlarge`

Mean of 2–4 runs,
whiskers ±1 sd.



Evaluation

Core Utilization

	miniOSv	Linux		AnyBlob	
budget	Gbps	Gbps	cores	Gbps	cores
2	71.6	43.0	2.00	40.0	1.58
4	109.3	80.2	4.00	68.0	2.39
8	157.7	148.7	7.97	121.3	3.36
16	162.9	169.1	13.46	161.1	4.46
32	164.5	168.0	16.27	163.7	5.62

Bulk GETs from nginx on `r6in.32xlarge`. miniOSv cores always equal the budget due to busy polling.

A Unikernel Network Stack

Related Work in TCP/IP Networking

Not Self-Contained	
Linux, FreeBSD	native stacks out because of code size
Z-Stack, F-Stack	reuse most of the FreeBSD network stack
TAS	relies on Linux for slow path
netmap	relies on Linux
Seastar	only works within its framework
VPP	host stack tied to the graph framework

Not Built for Throughput	
lwIP	optimized for resource utilization, not performance
smoltcp	good single-core performance, one poll loop per instance, no RSS

A Unikernel Network Stack

Implementation Effort

component	origin	LoC
ENA NIC driver	ported from OSv	~9 000
miniDPDK	ported from OSv	~1 000
mininet	new: workers, predictable RSS, connections	~4 000
DuckDB port	miniOSv runtime	~3 600
DuckDB → mininet	custom HTTP client for httpfs	~400

smoltcp and rustls are used unmodified. No optimizations inside DuckDB.

Limitations and Future Work

- **One flow, one core.** A connection is capped at per-core bandwidth. Fine for plain HTTP, but TLS lowers that ceiling. Fix: split requests across connections.
- **No GRO or other software offloads.** Without jumbo frames, per-packet cost dominates.
- **Static core allocation.** A fixed number of networking cores. miniOSv has no load balancer to resize it at runtime.
- **Not zero-copy.** Unmodified smoltcp and rustls trade copies for engineering effort.
- **Object store behavior.** No tuning of request sizes, no preference for S3 endpoints that performed well before.
- **Shallow engine integration.** Run-to-completion inside the engine would give it full control over the stack.